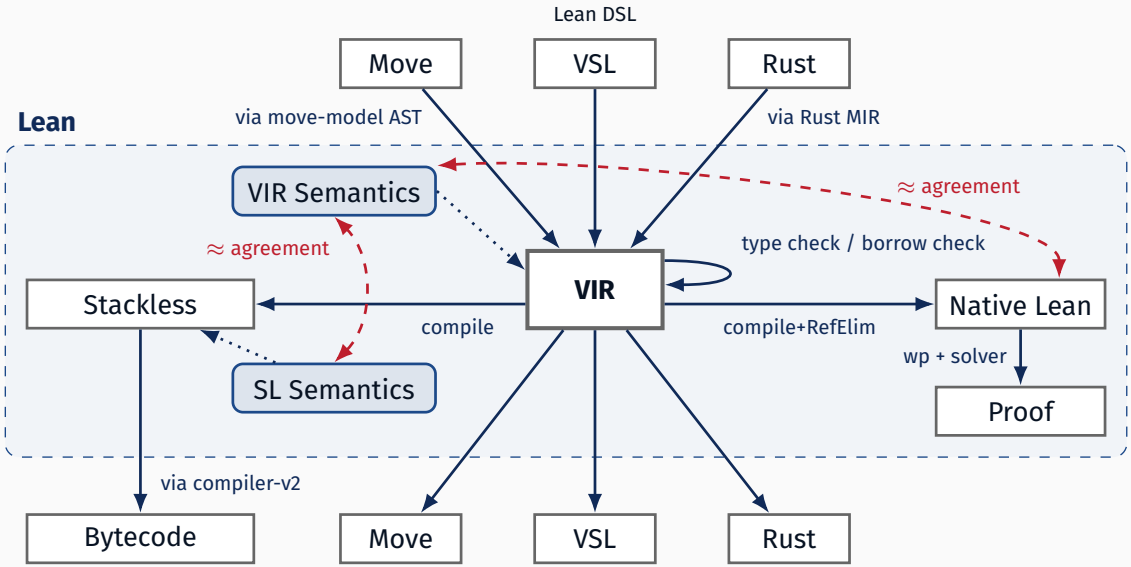
Verex Lean Stack

Architecture overview · Move Summit 2026 (internal)

Wolfgang Grieskamp

Aptos Labs

Architecture



Move sources are verified as VSL

Move

account.move

```
public entry fun deposit(addr: address,
                         amount: u64) {
  let value = &mut Balance[addr].balance.value;
  *value = *value + amount;
}
spec deposit {
  requires exists<Balance>(addr);
  modifies global<Balance>(addr);
  ensures Balance[addr].balance.value
    == old(Balance[addr].balance.value) +
     amount;
  aborts_if Balance[addr].balance.value
    + amount > MAX_U64;
}
```

VSL, rendered from Move

```
module 0x42::account where
  public entry fun deposit(addr : Address,
      amount : u64) -> Unit := do
    let value := &mut Balance[addr].balance.value
    *value := *value + amount

  spec deposit where
    requires exists<Balance>(addr)
    ensures global<Balance>(addr).balance.value
      == old(global<Balance>(addr).balance.value)
         + amount
    aborts_if global<Balance>(addr).balance.value
      + amount > MAX_U64
    modifies global<Balance>(addr)
```

One verifier for every source language: Move and Rust are rendered as VSL, and messages are mapped back to the original files.

Rust sources are verified as VSL

Rust

bounds.rs

```
pub fn max(left: u32, right: u32) -> u32 {
    if left ≥ right { left } else { right }
}

pub fn raise(slot: &mut u32, floor: u32) {
    if *slot < floor {
        *slot = floor;
    }
}
```

VSL specifications

bounds.spec.lean

```
spec max where
  ensures result ≥ left && result ≥ right
  ensures result == left || result == right

spec raise where
  ensures slot ≥ floor
  ensures slot == old(slot) || slot == floor
```

VSL, rendered from Rust

```
namespace bounds where
  fun max(left : u32, right : u32) -> u32 :=
    if left ≥ right then left else right

  spec max where
    ensures result ≥ left && result ≥ right
    ensures result == left || result == right

  fun raise(slot : &mut u32,
      floor : u32) -> Unit := do
    if *slot < floor then *slot := floor

  spec raise where
    ensures slot ≥ floor
    ensures slot == old(slot) || slot == floor
```

From the Move developer's view

```
$ move prove --lean --package-dir vault
```

Messages at their Move source positions:

```
vault.move:38:9: error: the specification
  clause `ensures result == ...` is not
  established
vault.move:61:13: error: the loop invariant
  `i < periods` is not preserved by an
  iteration
```

- Also `aptos move prove --lean` and the Move Prover unit tests (`MVP_TEST_FEATURE=lean`).
- Budget per function in heartbeats: `pragma heartbeats`.

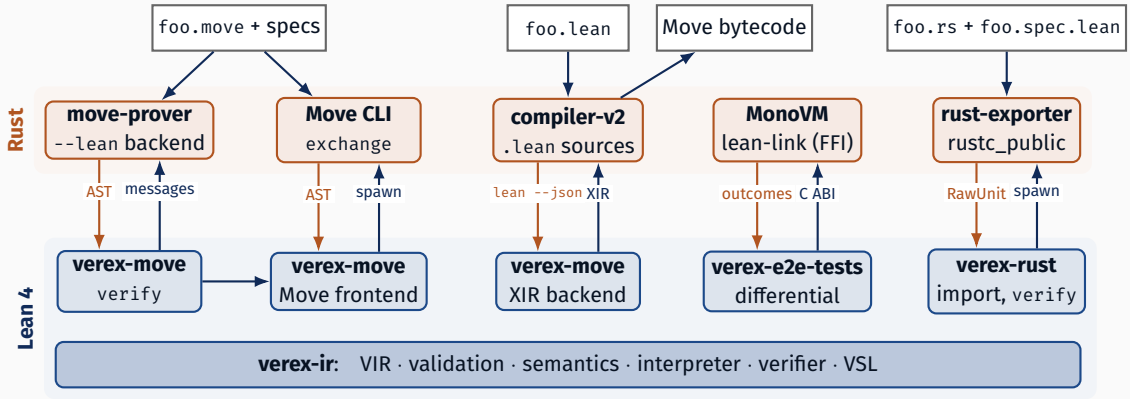
When automation does not finish, the proof goes in a file beside the source:

```
spec square_of_sum {
  pragma verify = manual; // in proofs.proof.lean
  ensures result == a * a + 2 * a * b + b * b;
}
```

```
-- proofs.proof.lean
verify square_of_sum by
  verex_obligations
  all_goals (first | verex_decide
    | (verex_prepare; all_goals try verex_decide))
  case ensures_1 =>
    rw [Int.add_mul, Int.mul_add, ...]
    omega
```

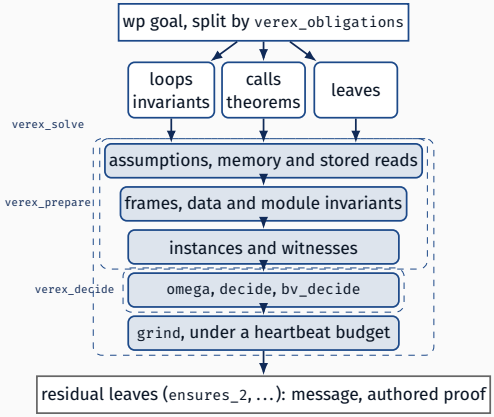
`verify f by verex_obligations` lists the obligations, one `case` per clause.

Lean and Move Stack Interplay



- Verify Move: checked AST out, messages back.
- Compile VSL: Lean emits XIR, compiler-v2 makes bytecode.
- Test: MonoVM linked into Lean test executables.
- Rust: borrow-checked MIR from rustc.

The Solver Tactics



- **Loops**: invariant at entry, preserved by an iteration, then the exit.
- **Calls**, as in the Move Prover: inlined unless native, opaque, or on a cycle; those go through contracts and theorems.
- **Leaves**, one per clause and path: `verex_prepare` normalizes, `verex_decide` runs the bounded deciders, `verex_solve` the full cascade, each under a budget.
- **Authored proofs**: Move `spec lemma` and `proof` blocks, or `verify f by` with the same tactics per clause (`case ensures_1 => ...`), then any Lean tactic.
- The solver is Lean meta code and *untrusted*; only its proof terms are checked.

What is proved, and what is trusted


```
theorem deposit.verified
    (prepared : prepareExecution registry unit
                  = .ok executable)
    (globalsPreserved : GlobalsPreserved executable) :
    SatisfiesFunction executable deposit
      deposit.contract
```

`SatisfiesFunction` reads the function's meaning off the big-step judgment. For every input meeting `requires`:

- every return meets `ensures` and the frame,
- every abort is one of `aborts_if`,
- no run is undefined.

Also:

- partial correctness.
- generic functions: proved once at a fresh type (monomorphization as in the Move Prover), valid at every instantiation.

Trusted

- Lean kernel, plus native `bv_decide` certificates.
- Big-step semantics of VIR as the meaning of Move/Rust.
- Frontend (AST $\rightarrow$ VIR) and contract translation.

Not claimed

- Anything about emitted bytecode; only up to stackless BC

Proofs TBD

- General agreement theorem, stated once over the compiler for every function.
- GlobalsPreserved: hypothesis, to be derived from type preservation.

	Move Prover	Verex Prover
Input	bytecode after the compiler	checked AST → validated VIR
Engine	Boogie, Z3 / CVC5	Lean tactics, Lean kernel
Result	verdict, counterexample	theorem
Semantics	implicit in the encoding	explicit big-step semantics, interpreter proved sound and complete
References	reference elimination	prophecies (RustHorn)
When stuck	timeout; restate the spec	message per clause; proof in `foo.proof.lean`
Specifications	MSL	the same MSL, plus `pragma heartbeats` and `verify = manual`

Status and next steps

Works today

- `move prove --lean` on packages; Move Prover unit tests run with the Verex Prover.
- Nightly benchmark of 32 problems from `move-stdlib`, `aptos-stdlib`, the framework, Rust: 24 verify completely.
- Generics, storage, loops, vectors, function values, invariants, lemmas, state labels (in part).
- compiler-v2 compiles VSL; MonoVM differential tests.
- Rust, early: safe code from MIR, `verex-rust verify` with specifications in `foo.spec.lean`.

Open

- Prove the agreement theorem; remove the last project axioms.
- Speed: the standard libraries in minutes.
- Coverage: intrinsic maps, remaining state labels, natives.
- Compiler correctness, VSL to bytecode.
- Rust: regions, drops and panics, unsafe code, library models.

Questions?

`third_party/move/lean`: `README.md`, `designs/`

Backup: VIR and semantic profiles

- VIR is language-neutral: functions carry a profile (Move, Rust).
- Profiles plug in twice:
  - validation: which types, operations, intrinsics, and storage a profile admits;
  - execution: how operations evaluate, what aborts or panics (Move abort codes, Rust `panic=abort`).
- The same semantics is run by the interpreter, defined by the big-step relation, and used by the verifier.
- Validated namespaces are relocatable: modules link into units without re-validation.

- A mutable reference denotes a pair: the current value and a *prophecy* of its final value (RustHorn).
- When the loan ends, the prophecy is resolved to the value then held.
- Validation computes where loans die; the interpreter, big-step relation, and verifier share this model.
- No heap or separation logic: Move's alias-free references and per-resource memory make this enough, also for Rust's safe subset.